

Health Memory — from a persistent record to a record the patient can speak to, in their own language, and control

AI-Agent-Powered Persistent Health Memory Layer for Elderly Patients

APPLICANT Suresh Kumar K	DEPARTMENT Information Technology	INSTITUTION Saveetha Engineering College (Autonomous), Chennai	DATE 27 September 2026
-----------------------------	--------------------------------------	---	---------------------------

01 The original prototype, and exactly what is being added

What was submitted at Stage 1

The prototype is a persistent health-memory layer for elderly patients. An elderly patient's medical history is normally scattered across paper prescriptions, several hospitals, a caregiver's recollection and a family member's phone. The prototype gives that history one home that never forgets and never loses provenance.

Information enters through three routes — typed text, spoken entry, or a photographed document. Scanned records pass through a handwriting-aware OCR stage that scores every extracted field. Fields below the confidence threshold 0.85 that are also clinically important (drug name, dose, frequency, allergy, critical instruction) are never written silently into the record: they are routed to a human verification queue. Everything stored carries who said it, when, from what source, at what confidence, and under which consent scope. Four role dashboards — Patient/Guardian, Doctor, Caregiver/Old Age Home, and the verification role — read the same memory through a consent-aware retrieval pipeline, each with a role-specific assistant that answers only from records that role is permitted to see.

What is new since that application

Three additions, built directly onto the existing codebase rather than as a new project. Together they address the gap between *a system that works* and *a system an elderly Tamil-speaking patient in Chennai can actually use and trust*.

NEW · A Bilingual Tamil-English interface Every interface string in both languages, switchable anywhere including before login.	NEW · B Tamil and mixed-speech voice intake Speak Tamil, English, or both in one sentence; an assisted English reading is derived.	NEW · C Patient-controlled care team The patient, not the doctor, decides which doctors may see the record.	NEW · D Document data layer The store moved from a single-file relational database to MongoDB collections.
--	---	--	---

A · The bilingual layer

926 interface strings are now available in English and தமிழ். The switch sits in the dashboard top bar, the public navigation and both authentication screens, so a patient or a guardian chooses the language *before* they have to read anything. Dates and relative times follow the choice. Clinical content — medication names, memory-event text, OCR output, AI-generated drafts, anything a clinician wrote — is deliberately never translated; only interface chrome is.

B · Voice that accepts how people actually speak

The voice diary now offers three speaking modes: English, தமிழ், and தமிழ் + English. A new language service detects the script mix of any patient entry — typed or spoken — and derives a plain-English *assisted reading* so that extraction can work and an English-reading clinician can follow the entry:

Input (Tamil speech): எனக்கு இரண்டு நாளாக தலை வலிக்குது

Assisted reading: *I have had a headache for two days.*

Stored as: the patient's original Tamil wording, event type SYMPTOM, trust level Patient Reported, confidence capped at 0.88 — **not** a diagnosis.

The reading is produced by a reviewable phrase table, not a translation service. Every mapping is a fixed, inspectable pair covering the vocabulary elderly patients actually use — symptoms, body parts, durations, medicine and visit language. Anything outside the table stays in the patient's own words rather than being guessed at.

C • Consent extended from data to relationship

Previously a doctor named in an uploaded prescription was linked to the patient automatically. That is convenient and wrong: it makes the record's reach a side effect of paperwork. Now the patient owns the relationship. They search a directory, add a doctor, hold several at once, nominate a primary, and remove any of them. Adding a doctor is what places the patient on that doctor's dashboard; removing takes them off it. A doctor named in a scanned record is *suggested* to the patient and receives nothing until the patient accepts. There is no code path anywhere in the backend by which a doctor can add, remove or alter a care relationship — including their own.

Removal withdraws access; it never deletes history. The relationship is marked removed and retained. Every memory event, prescription, medication, consultation, lab result, diagnosis, hospital visit and document that recorded that doctor still points at them, and the doctor remains listed as a past doctor with a count of the records they contributed — so the patient can see what that clinician gave them, and add them back later.

D • Supporting change: document data layer and role generalisation

The store moved from a single-file relational database to MongoDB, with the patient–doctor relationship modelled as a join collection that supports many doctors per patient and many patients per doctor, updated live as the patient edits their care team. The verification role was also generalised from *Pharmacist* to *Reviewer*, since the function is confirming low-confidence extracted data — work a pharmacist, a nurse or a clinical records officer can each do — while keeping that dashboard's behaviour and appearance identical.

02 How this improves the work, and why the approach is smarter and cheaper

What the update changes about the product

Stage 1 proved the record could be trusted. This stage makes it reachable. A 72-year-old in Chennai who is more fluent speaking Tamil than reading English could previously have a perfect health memory built *about* them by their guardian; they can now build it themselves, by speaking one sentence in the language they think in. And the person whose body the record describes now decides who reads it.

Against what exists today

APPROACH IN THE MARKET	WHERE IT STOPS	WHAT THIS PROJECT DOES INSTEAD
Hospital EMR / HIS	Organised around one institution's episodes. The record does not follow the patient between hospitals, and the caregiver and family are not users at all.	Organised around the patient's lifetime, with caregiver and guardian as first-class roles.
National health-ID record linkage	Solves discovery and exchange of documents. It does not turn scattered documents into a usable memory, and it does not help someone enter what happened in their own words.	Complements it: an intake and memory layer, with vernacular entry and provenance on every fact.
Consumer health apps	Transactional — booking, ordering, teleconsult. History is a by-product, largely English, and the patient rarely controls onward visibility.	History is the product; language and relationship control are the interface.
Cloud translation / LLM for vernacular support	Per-call cost that scales with every elderly user; protected health information leaves the deployment; a generative translator can silently produce a clinical claim the patient never made.	A fixed, auditable phrase table plus an English-keyed dictionary. No per-call cost, no network dependency, no PHI egress, and it cannot invent a symptom.

Why the engineering choices are the affordable ones

- **Translation as data, not as a service.** The dictionary is keyed by the English source string itself, so there are no invented key names to maintain and a missing entry renders in English rather than as a blank or a raw key. A translation gap degrades to readable English instead of a broken screen — which is what makes it safe for a vocabulary maintained by clinicians and volunteers rather than by a vendor.

- **Determinism where safety is at stake.** The same property that makes a phrase table less fluent than a neural translator is what makes it acceptable here: its output is reviewable in advance, identical every time, and incapable of adding meaning. The reading is explicitly an aid — the stored truth is the patient's own words, the confidence is capped, and the entry enters the same human-review workflow as any other patient statement.
- **No paid inference in the critical path.** OCR runs locally, retrieval uses locally-computed embeddings, and the assistants fall back to a deterministic grounded provider when no model key is configured. The system is fully functional offline. Recurring cost is a small virtual machine and a database — not a per-request bill that grows with every patient served.
- **A migration that cost almost nothing to verify.** Moving to MongoDB by rewriting all 143 query sites would have put the clinical logic — confidence routing, consent filtering, retrieval ranking — back on the table. Replacing the mapper underneath them instead left every service, route and agent module byte-identical, so the behaviour was not merely re-tested but structurally unchanged.

03 Testing: the biggest insight, the hurdle, and what happens next

What was tested, and how

Testing at this stage is automated and browser-level, not yet with patients. Two suites run against the real system: an end-to-end suite driving the HTTP API, and a browser suite driving every dashboard in both languages at desktop and phone widths.

62	45	926	457	0
API checks passing, 0 failing	Browser checks passing, 0 console errors	Interface strings in both languages	Records migrated, 0 fields unmapped	Records lost when a doctor is removed

BEHAVIOUR VERIFIED	OBSERVED RESULT
Patient adds a doctor	That doctor's patient list grows from 2 to 3; the patient appears on their dashboard immediately.
Doctor attempts to change the relationship	Add, remove, set-primary and even read all refused with HTTP 403.
Patient removes a doctor	Doctor loses access (403) and the patient leaves their list; 22 memory events and 4 medications remain intact; the removed doctor is retained with the 12 records they contributed.
Tamil speech entry	Detected as Tamil; reading produced exactly as specified; stored as SYMPTOM / Patient Reported, never a diagnosis; original Tamil preserved verbatim.
Mixed Tamil–English speech	Detected as mixed; English medical terms embedded in Tamil speech retained rather than discarded.
Language switching across all four dashboards	No layout shift, no horizontal overflow at 1440 px or 390 px, choice persists across reload.

The biggest insight

The hard problem in vernacular healthcare software is not translating text. It is deciding what must *not* be translated, and being honest about how good the translation is.

Two findings drove that conclusion. First, the boundary between interface and clinical content turned out to be the real design work: a drug name, an OCR reading or a doctor's note rendered into Tamil is no longer evidence of what was written, so the system now treats clinical content as data that is displayed verbatim in either language, and translates only the chrome around it.

Second, and less comfortably: word-by-word mapping between Tamil and English produces grammatical nonsense on free-form sentences, because Tamil is subject–object–verb and English is subject–verb–object. The high-frequency clinical pattern renders perfectly; an unconstrained sentence does not. The honest output of the current system, shown unedited:

CONSTRAINED CLINICAL PATTERN — WORKS

எனக்கு இரண்டு நாளாக தலை வலிக்குது
→ "I have had a headache for two days."

FREE SENTENCE — DEGRADES VISIBLY

நேற்று டாக்டர் Arun Kumar-ஐ பார்த்தேன், அவர் என் blood pressure மாத்திரை மாற்றினார்
→ "Yesterday doctor Arun Kumar saw, he my blood pressure tablet changed."

That second output is understandable but clumsy, and the design response matters more than the clumsiness: rather than hide it behind a fluent generative translator that might smooth the sentence into a claim the patient did not make, the reading is labelled as assisted, its confidence is capped, and the patient's original wording remains the stored record. A clinician reading it can see both. Fluency was traded for the guarantee that nothing was invented.

The unexpected hurdle

Internationalisation turned out to be a *layout* problem as much as a language problem. Tamil labels run considerably longer than their English equivalents, and the public navigation — which fits comfortably in English at 1440 px — overflowed as soon as Tamil was selected. The fix was a breakpoint scoped to the active language, collapsing the link row into the existing menu earlier in Tamil only, leaving the English rendering untouched. The general lesson: a bilingual retrofit needs a visual regression pass in both languages at every breakpoint, not just a string audit.

A second hurdle was the database migration. The obvious route — rewriting every query to the new driver — would have been idiomatic and risky, touching the confidence-routing and consent-filtering logic that the whole safety argument rests on. Replacing the mapper beneath those queries instead was the smaller and far safer change, and the full suite passed unchanged afterwards.

Immediate next steps toward real users

#	STEP	WHAT IT INVOLVES AND WHY IT IS NEXT
1	Clinical and linguistic review of the Tamil vocabulary	A Tamil-speaking nurse and a geriatrician review all 926 entries and the phrase table. Machine-assisted translation is adequate for a demonstration and not for a medical interface; this is the gate before any patient sees it.
2	Supervised field pilot at an old age home	A small cohort of residents, with a caregiver present, using voice intake in Tamil. Primary measures: proportion of spoken entries that need no correction, time to record one entry, and reviewer-queue throughput.
3	Widen the phrase table from real speech	The pilot transcripts, not our imagination, decide which patterns to add. Any future model-based reading sits behind the same guardrail — assisted, capped, never a diagnosis — with the audited table kept as the primary path.
4	Production database deployment	Provision a managed MongoDB instance, disable the local development fallback, and rehearse backup and restore with the migration script against a realistic data volume.
5	Accessibility pass in both languages	Screen-reader walkthrough in Tamil, translation of the remaining illustration descriptions, and type-scale testing with elderly participants.
6	Security review and data-protection documentation	Independent review of authentication, the consent gate and the audit trail, with handling documented against India's data-protection obligations before any real patient record is stored.
7	Interoperability study	Assess alignment with the national health-ID ecosystem so the memory layer complements rather than duplicates record linkage.

All figures above are reproducible from the repository: `python seed.py` & `python e2e_check.py` for the API suite, and the browser suite against the running application. All patient data used in testing is fictional.